Practical No.: 04

NAME: Bhalerao Tanvi Vitthal
ROLLNO.: 08
COURSE: AI&DS
CLASS: BE
SUB: Computer Laboratory-IV
Title: Image Classification Using Convolutional Neural Networks (CNNs) with Hyperparameter Optimization

0.1 Import Required Libraries

[2]:
```
import tensorflow as tf
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense,␣
 ↪Dropout
from tensorflow.keras.optimizers import Adam, SGD
from tensorflow.keras.utils import to_categorical

import matplotlib.pyplot as plt
import numpy as np
```

0.2 Load Dataset

[3]:
```
(x_train, y_train), (x_test, y_test) = cifar10.load_data()

print("Training shape:", x_train.shape)
print("Testing shape:", x_test.shape)
```

```
Training shape: (50000, 32, 32, 3)
Testing shape: (10000, 32, 32, 3)
```

0.3 Data Preprocessing

[4]:
```
# Normalize pixel values
x_train = x_train / 255.0
x_test = x_test / 255.0

# One-hot encode labels
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
```

0.4 Define CNN Model (with Tunable Hyperparameters)

[5]:
```
def create_cnn_model(
    filters1=32,
    filters2=64,
    kernel_size=(3,3),
    dropout_rate=0.5,
    learning_rate=0.001,
    optimizer_type="adam"
):
    model = Sequential()

    model.add(Conv2D(filters1, kernel_size, activation='relu',␣
 ↪input_shape=(32,32,3)))
    model.add(MaxPooling2D((2,2)))

    model.add(Conv2D(filters2, kernel_size, activation='relu'))
    model.add(MaxPooling2D((2,2)))

    model.add(Flatten())
    model.add(Dense(128, activation='relu'))
    model.add(Dropout(dropout_rate))
    model.add(Dense(10, activation='softmax'))

    if optimizer_type == "adam":
        optimizer = Adam(learning_rate=learning_rate)
    else:
        optimizer = SGD(learning_rate=learning_rate, momentum=0.9)

    model.compile(
        optimizer=optimizer,
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    return model
```

0.5 Train Baseline CNN Model

[6]:
```
baseline_model = create_cnn_model()

history_baseline = baseline_model.fit(
    x_train,
    y_train_cat,
    epochs=10,
    batch_size=64,
    validation_split=0.2
```

```
)
```

D:\Old ssd\Anaconda3\Anaconda\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
  super().__init__(activity_regularizer=activity_regularizer,    **kwargs)
```

```
Epoch 1/10
625/625                24s 34ms/step -
accuracy: 0.3636 - loss: 1.7317 - val_accuracy: 0.4973 - val_loss: 1.4252
Epoch 2/10
625/625                40s 32ms/step -
accuracy: 0.4800 - loss: 1.4403 - val_accuracy: 0.5393 - val_loss: 1.2988
Epoch 3/10
625/625                21s 33ms/step -
accuracy: 0.5321 - loss: 1.3152 - val_accuracy: 0.5719 - val_loss: 1.2132
Epoch 4/10
625/625                22s 35ms/step -
accuracy: 0.5634 - loss: 1.2317 - val_accuracy: 0.6170 - val_loss: 1.1113
Epoch 5/10
625/625                21s 34ms/step -
accuracy: 0.5864 - loss: 1.1728 - val_accuracy: 0.6388 - val_loss: 1.0416
Epoch 6/10
625/625                22s 34ms/step -
accuracy: 0.6073 - loss: 1.1099 - val_accuracy: 0.6388 - val_loss: 1.0271
Epoch 7/10
625/625                21s 34ms/step -
accuracy: 0.6187 - loss: 1.0783 - val_accuracy: 0.6453 - val_loss: 1.0033
Epoch 8/10
625/625                21s 33ms/step -
accuracy: 0.6358 - loss: 1.0266 - val_accuracy: 0.6670 - val_loss: 0.9500
Epoch 9/10
625/625                22s 34ms/step -
accuracy: 0.6500 - loss: 0.9933 - val_accuracy: 0.6665 - val_loss: 0.9602
Epoch 10/10
625/625                20s 33ms/step -
accuracy: 0.6622 - loss: 0.9590 - val_accuracy: 0.6690 - val_loss: 0.9425
```

0.6 Evaluate Baseline Model

[7]:
```
baseline_loss, baseline_acc = baseline_model.evaluate(x_test, y_test_cat)
print("Baseline Test Accuracy:", baseline_acc)
```

```
313/313                2s 8ms/step -
accuracy: 0.6678 - loss: 0.9477
Baseline Test Accuracy: 0.6678000092506409
```

0.7 Optimized CNN Model (Tuned Hyperparameters)

[8]:
```
optimized_model = create_cnn_model(
    filters1=64,
    filters2=128,
    kernel_size=(3,3),
    dropout_rate=0.3,
    learning_rate=0.0005,
    optimizer_type="adam"
)

history_optimized = optimized_model.fit(
    x_train,
    y_train_cat,
    epochs=15,
    batch_size=64,
    validation_split=0.2
)
```

```
Epoch 1/15
625/625                 56s 85ms/step -
accuracy: 0.4021 - loss: 1.6486 - val_accuracy: 0.5063 - val_loss: 1.3733
Epoch 2/15
625/625                 53s 85ms/step -
accuracy: 0.5365 - loss: 1.3041 - val_accuracy: 0.5763 - val_loss: 1.2225
Epoch 3/15
625/625                 60s 97ms/step -
accuracy: 0.5888 - loss: 1.1648 - val_accuracy: 0.6326 - val_loss: 1.0620
Epoch 4/15
625/625                 72s 81ms/step -
accuracy: 0.6252 - loss: 1.0726 - val_accuracy: 0.6544 - val_loss: 0.9979
Epoch 5/15
625/625                 84s 134ms/step -
accuracy: 0.6469 - loss: 1.0106 - val_accuracy: 0.6672 - val_loss: 0.9718
Epoch 6/15
625/625                 68s 108ms/step -
accuracy: 0.6704 - loss: 0.9469 - val_accuracy: 0.6733 - val_loss: 0.9330
Epoch 7/15
625/625                 72s 91ms/step -
accuracy: 0.6870 - loss: 0.8933 - val_accuracy: 0.6980 - val_loss: 0.8807
Epoch 8/15
625/625                 103s 126ms/step -
accuracy: 0.7049 - loss: 0.8463 - val_accuracy: 0.6863 - val_loss: 0.8957
Epoch 9/15
625/625                 84s 128ms/step -
accuracy: 0.7174 - loss: 0.8004 - val_accuracy: 0.7018 - val_loss: 0.8542
Epoch 10/15
625/625                 85s 132ms/step -
```

```
accuracy: 0.7363 - loss: 0.7568 - val_accuracy: 0.7037 - val_loss: 0.8450
Epoch 11/15
625/625                 159s 159ms/step -
accuracy: 0.7492 - loss: 0.7133 - val_accuracy: 0.7123 - val_loss: 0.8332
Epoch 12/15
625/625                 139s 154ms/step -
accuracy: 0.7634 - loss: 0.6750 - val_accuracy: 0.7071 - val_loss: 0.8582
Epoch 13/15
625/625                 141s 151ms/step -
accuracy: 0.7755 - loss: 0.6362 - val_accuracy: 0.7188 - val_loss: 0.8162
Epoch 14/15
625/625                 141s 150ms/step -
accuracy: 0.7881 - loss: 0.6017 - val_accuracy: 0.7211 - val_loss: 0.8075
Epoch 15/15
625/625                 142s 149ms/step -
accuracy: 0.7979 - loss: 0.5691 - val_accuracy: 0.7206 - val_loss: 0.8446
```

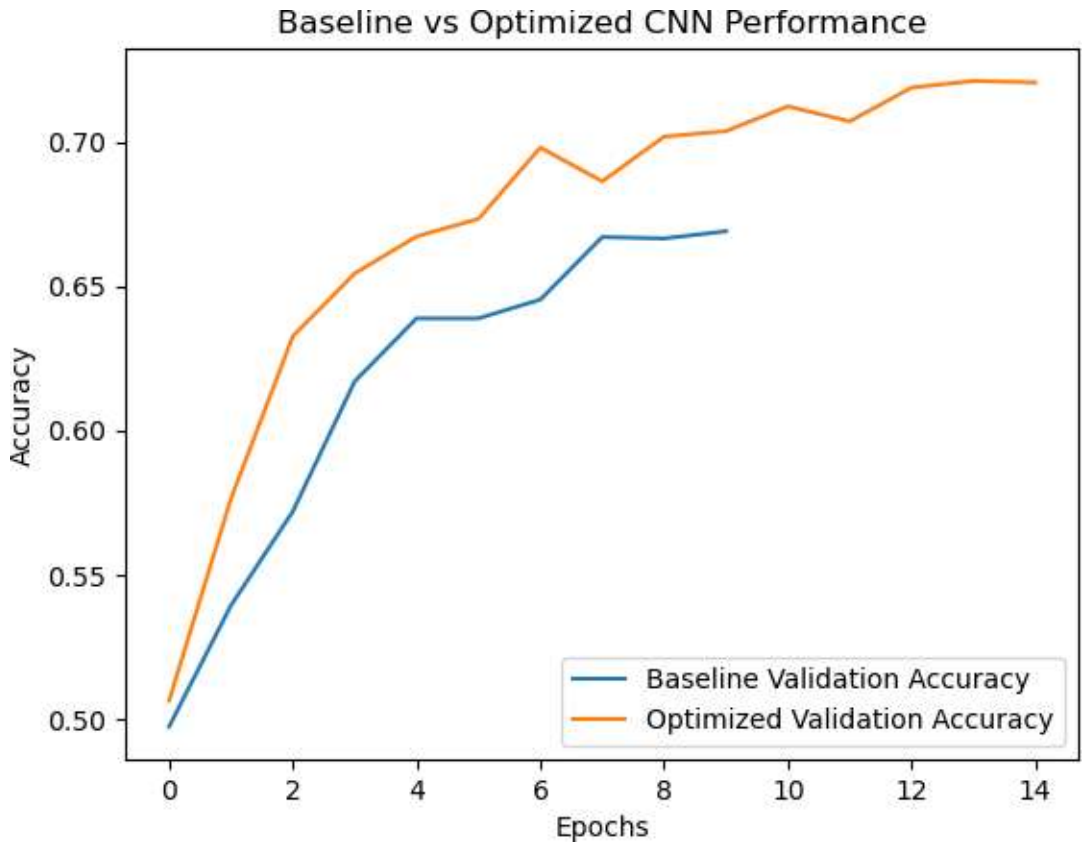
0.8 Evaluate Optimized Model

[9]:
```
opt_loss, opt_acc = optimized_model.evaluate(x_test, y_test_cat)
print("Optimized Test Accuracy:", opt_acc)
```

```
313/313                 7s 22ms/step -
accuracy: 0.7165 - loss: 0.8665
Optimized Test Accuracy: 0.7164999842643738
```

0.9 Accuracy Comparison Plot

[10]:
```
plt.plot(history_baseline.history['val_accuracy'], label='Baseline Validation␣
 ↪Accuracy')
plt.plot(history_optimized.history['val_accuracy'], label='Optimized Validation␣
 ↪Accuracy')
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.title("Baseline vs Optimized CNN Performance")
plt.show()
```


Baseline vs Optimized CNN Performance
Accuracy
0.70
0.65
0.60
0.55
0.50
0
2
4
6
8
10
12
14
Epochs
Baseline Validation Accuracy
Optimized Validation Accuracy